Table of Contents

Introduction

This document contains some notable sample prompts queried as part of the AnyLogic webinar on ChatGPT; it includes prompts both shown in the webinar and some that were cut for time/complexity constraints.

Each section represents a different topic, containing some commentary followed by one or more tables; each table represents a separate conversation (essentially a “clean slate”).

When reviewing, a few important things to keep in mind:

- Replicating these may produce responses different from what’s shown, likely due to a mixture of some randomness built into ChatGPT and updates to it over time.
- A mix of ChatGPT 3.5 and 4 was used. In short, 4 is vastly superior but requires a paid subscription to use and had usage limits at the time of testing (which is why there was some mixture). You can identify which by their icon – green for 3.5  and black for 4  .
- There are likely vastly better ways to communicate relevant information to these AIs. The style and tone used in the prompts were just based on personal intuition.

Basics of AnyLogic

Here, we explore how much ChatGPT knew about the technical process of building models in AnyLogic. Notable accuracies are highlighted in yellow and inaccuracies are in red.

As can be seen, it will confidently give incorrect information, which is why it's important to validate anything it gives you.

What do you know about building simulation models using the AnyLogic software? Particularly anything about how models are built within the interface

AnyLogic is a popular simulation modeling software that allows users to create, analyze, and experiment with various types of simulation models. It supports three main modeling paradigms: System Dynamics, Discrete Event, and Agent-Based Modeling. This flexibility allows users to combine different modeling techniques to address a wide range of problems.

Building models within the AnyLogic interface involves the following steps:

1. Create a new model: Start by creating a new project and selecting the desired modeling paradigm or combination of paradigms.
2. Design the model structure: Use the visual interface to design your model structure by adding elements such as agents, stocks, flows, events, and state charts. You can also define the connections and relationships between these elements.
3. Define model logic: In AnyLogic, you can define model logic using a combination of visual elements and code. You can use Java to write custom code for complex logic or to create reusable elements such as functions and libraries.
4. Parameterize the model: Define the parameters and initial values of your model. This may include setting agent populations, initializing stocks, or specifying distributions for random events. These parameters can be adjusted later to conduct sensitivity analyses or test alternative scenarios.
5. Create animations and user interfaces: AnyLogic offers a rich set of tools for creating animations and interactive user interfaces. This allows you to visualize and interact with your model as it runs, making it easier to understand the dynamics and identify potential issues.
6. Run simulations and analyze results: Once your model is complete, you can run simulations to generate data and analyze the results. AnyLogic provides various tools for data analysis and visualization, including charts, histograms, and heat maps. You can also export the data to external tools like Excel or R for further analysis.
7. Validate and refine the model: Based on the simulation results, you may need to validate your model by comparing it to real-world data or expert opinions. If necessary, refine the model by adjusting its parameters, structure, or logic.
8. Experiment with scenarios: After validating your model, you can use it to test various scenarios, conduct sensitivity analyses, or optimize decision-making processes.

The AnyLogic interface is designed to be intuitive and user-friendly, with drag-and-drop functionality,

context-sensitive menus, and a rich set of built-in libraries. To get started with building simulation models in AnyLogic, it is recommended to explore the software documentation, tutorials, and example models available on the AnyLogic website and community forums.

I have an established model with an Order agent type. I want to add a variable to it that starts at 0 and increments every time the agent enters a flowchart block. Walk me through the steps do implement this.

To add a variable to your Order agent type and increment it every time the agent enters a flowchart block, follow these steps:

1. Open your AnyLogic model and locate the Order agent type in the Project Browser. Double-click on the Order agent to open its properties.

2. In the Order agent properties, switch to the "Variables" tab. Click on the "Add" button to create a new variable. Name the variable, for example, "blockCounter", and set its type to "int" (integer). Set the initial value to 0.

3. Now, locate the flowchart block in your model where you want to increment the blockCounter variable. If you don't have a flowchart block yet, you can add one from the "Process Modeling Library" palette. Drag and drop the desired block, such as "Delay" or "Queue", onto your canvas.

4. Open the properties of the flowchart block by double-clicking on it. Switch to the "Actions" tab.

5. Find the "On enter" action. This action is triggered when an agent enters the flowchart block. In the "On enter" action code field, write the following code to increment the blockCounter variable of the Order agent:

```
agent.blockCounter++;
```

This code assumes that the agent entering the flowchart block is of type Order. The `agent` keyword refers to the agent entering the block, and `blockCounter++` increments the variable by 1.

6. Click "OK" to close the flowchart block properties.

7. Save your model and run a simulation to test the implementation. You can add a monitor or a text element to visualize the value of the blockCounter variable during the simulation.

These steps should help you add a variable to your Order agent and increment it every time the agent enters a specific flowchart block. You can repeat steps 3-5 for any additional flowchart blocks where you want the blockCounter to increment.

Basics of Java

This is an expanded version of what was shown in the webinar. It demonstrates not only ChatGPT's ability to contextually explain advanced topics in a specified tone, but the conversational aspect to it, allowing you to ask for follow-up questions and examples of what it describes.

I'm not a technical person but I'm using simulation software that makes use of Java. Can you explain in practical terms what the following are: Variables, functions, collections, arraylist, primitive arrays, classes, imports

Sure, I'll try to explain these concepts in a non-technical way so that you can understand how they relate to your simulation software.

1. Variables: Think of variables like containers or boxes that store information. They hold values such as numbers, text, or even more complex data. In your simulation software, variables could represent things like the speed of an object, the name of a character, or the position of an object in the simulation.

2. Functions: Functions are like recipes or sets of instructions that perform specific tasks. They often take some input (called arguments) and return a result. Functions can help break down complex tasks into smaller, more manageable steps. In your simulation software, functions might be used to calculate the distance between two objects, update the state of the simulation, or display information on the screen.

3. Collections: Collections are data structures that hold multiple pieces of information together, like a shopping list or a photo album. They can store multiple variables and are useful for organizing and processing data. In your simulation software, collections might be used to store a list of objects in the simulation or to keep track of events that have occurred.

4. ArrayList: An ArrayList is a type of collection in Java that stores elements in a dynamic, resizable array. It is similar to a list where you can add, remove, or access elements at specific positions. In your simulation software, ArrayLists might be used to keep track of a group of objects, such as the characters in a scene or the elements in a user interface.

5. Primitive arrays: Primitive arrays are similar to ArrayLists but store simpler, fixed-size data types, like integers or characters. They are more memory-efficient than ArrayLists and can be faster for certain operations due to their simplicity. They are declared using the data type followed by square brackets, such as int[] or char[].

I keep hearing the phrase about "an instance of an object or type", what does that mean?

"An instance of an object or type" refers to a specific occurrence or example of a particular class or data type in programming.

In more relatable terms, think of a class as a blueprint or a template for creating objects. An object is a specific example or instance created from that blueprint. Each object created from the same class can have different values for its properties, but they share the same structure and behavior defined by

the class.

For example, let's say you have a class called "Car" that defines the blueprint for a car. The class might contain information about the car's make, model, color, and maximum speed, as well as functions to start the engine or honk the horn.

Now, if you create a specific car, like a red Toyota Corolla, you're creating an instance of the "Car" class. This particular car (the instance) would have the specific properties of being a red Toyota Corolla, but it would still have the same structure and behavior as defined by the "Car" class (i.e., it can start the engine or honk the horn).

In your simulation software, instances of objects might represent the individual elements of the simulation, such as characters, vehicles, or buildings. Each instance would have its own unique properties but would follow the same structure and behavior defined by their corresponding classes.

Can you give an example of the Car? Only use snippets, don't write a full program

Sure, here are some code snippets to demonstrate the "Car" example:

1. Defining the Car class:

```
public class Car {
    // Properties of the car
    String make;
    String model;
    String color;
    int maxSpeed;

    // Constructor to create an instance of the car with specific values
    public Car(String make, String model, String color, int maxSpeed) {
        this.make = make;
        this.model = model;
        this.color = color;
        this.maxSpeed = maxSpeed;
    }

    // Function to start the engine
    public void startEngine() {
        System.out.println("The engine is started.");
    }

    // Function to honk the horn
    public void honk() {
        System.out.println("Honk! Honk!");
    }
}
```

2. Creating an instance of the Car class:

```
Car myCar = new Car("Toyota", "Corolla", "Red", 120);
```

3. Accessing properties and using functions of the instance:

```
System.out.println("Make: " + myCar.make);
System.out.println("Model: " + myCar.model);

myCar.startEngine();
myCar.honk();
```

These snippets demonstrate defining the Car class, creating an instance of it (a red Toyota Corolla), and using its properties and functions. Note that these are not complete programs, just parts to help illustrate the concept.

When you create the object, why do you need to start it with “Car” if you’re obviously creating a Car object with the “new Car” part?

When you create an object in Java, you need to specify the type of the object before the variable name. This is because Java is a statically-typed language, meaning that the type of each variable must be explicitly declared at the time of initialization.
In the example I provided, “Car” is the type of the object you’re creating, and “myCar” is the name of the variable that will hold a reference to the new object. The “new Car()” part is the actual creation of the object using the constructor, which initializes the object with specific values.

Here’s the line of code for creating the Car object again:

```
Car myCar = new Car("Toyota", "Corolla", "Red", 120);
```

Breaking it down:

- `Car` : The type of the object you’re creating, which is the Car class.
- `myCar` : The variable name that will hold a reference to the newly created Car object.
- `=` : The assignment operator, which assigns the newly created object to the variable.
- `new Car("Toyota", "Corolla", "Red", 120)` : The actual creation of the Car object using the constructor with specific values.

Declaring the type before the variable name allows the Java compiler to check for type-related errors at compile time, making the code more robust and less prone to errors. This also provides better code readability and understanding, as it’s clear what type of object is being created and what methods and properties it should have.

What’s the difference between calling a variable or function from the type versus the object? How do I know when to use which one

In Java, you can have variables and functions associated with the class (type) itself, or with instances (objects) of the class. The main difference is whether they are declared as `static` or not.

1. Variables and functions associated with the class (type) are declared as `static`. They belong

to the class itself, not to individual instances. You can access them using the class name followed by a dot (`.`) and the variable or function name. Since they are shared by all instances of the class, any change made to a static variable will affect all instances.

2. Variables and functions associated with instances (objects) are non-static. They belong to each individual object created from the class, and each object has its own copy of these variables and functions. You access them using the object (instance) name followed by a dot (`.`) and the variable or function name.

Here's an example to illustrate the difference:

```
public class MyClass {
    // A static variable (associated with the class/type)
    public static int staticVar = 0;

    // A non-static variable (associated with instances/objects)
    public int instanceVar = 0;

    // A static function (associated with the class/type)
    public static void staticFunction() {
        System.out.println("Called from the class");
    }

    // A non-static function (associated with instances/objects)
    public void instanceFunction() {
        System.out.println("Called from an instance");
    }
}
```

To access the static variable and function, you would use the class name:

```
MyClass.staticVar = 10;
MyClass.staticFunction();
```

To access the non-static variable and function, you would first create an instance of the class and then use the instance name:

```
MyClass myObject = new MyClass();
myObject.instanceVar = 10;
myObject.instanceFunction();
```

To know when to use which one, consider the following:

- If the variable or function is related to the class as a whole and should be shared among all instances, use a `static` variable or function.
- If the variable or function is related to individual instances and should have separate values or behaviors for each instance, use a non-static variable or function.

Explain for-loop and what a "Stream" is

1. For-loop: A for-loop is a programming construct that allows you to execute a block of code repeatedly for a specific number of iterations. It is commonly used to iterate over a range of values, a collection, or an array. A for-loop has three main components: the initialization, the condition, and the increment.

Here's a simple example of a for-loop in Java:

```
for (int i = 0; i < 5; i++) {
    System.out.println("Iteration: " + i);
}
```

- Initialization: `int i = 0;` This part sets the initial value of the loop control variable (`i` in this case). It is executed only once, at the beginning of the loop.
- Condition: `i < 5;` This part is a boolean expression that is checked before each iteration. If it evaluates to `true` , the loop continues; if it evaluates to `false`, the loop stops.
- Increment: `i++` This part updates the loop control variable after each iteration (in this case, incrementing `i` by 1).

The above loop will print the numbers 0 to 4, iterating 5 times in total.

2. Stream: In Java, a Stream is a high-level, functional programming abstraction introduced in Java 8. It represents a sequence of elements that can be processed in parallel or sequentially. Streams provide a convenient and efficient way to perform operations like filtering, mapping, or reducing on collections, arrays, or other data sources.

Streams are designed to work with lambda expressions, which are concise, anonymous functions that can be used to express instances of single-method interfaces.

Here's an example of using a Stream with a List of integers to filter even numbers and print them:

```
import java.util.Arrays;
import java.util.List;
import java.util.stream.Stream;

public class StreamExample {
    public static void main(String[] args) {
        List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5, 6, 7, 8, 9, 10);

        Stream<Integer> evenNumbersStream = numbers.stream().filter(n -> n % 2 ==
0);
        evenNumbersStream.forEach(System.out::println);
    }
}
```

In this example, we create a List of integers, then use the `stream()` method to create a Stream from the List. We apply a filter operation to keep only the even numbers and then use the `forEach` method to print them.

Note that Streams are not meant to replace loops in all scenarios, but they can offer a more concise and functional way to perform operations on sequences of data in some cases.

Can you show the above example but with a loop? Also, please explain when you'd want to use a stream vs loop and why

Sure, here's the same example using a for-loop to filter even numbers and print them:

```java
import java.util.Arrays;
import java.util.List;

public class LoopExample {
    public static void main(String[] args) {
        List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5, 6, 7, 8, 9, 10);

        for (int number : numbers) {
            if (number % 2 == 0) {
                System.out.println(number);
            }
        }
    }
}
```

In this example, we use an enhanced for-loop (also known as a "for-each" loop) to iterate over the `numbers` list. For each number, we check if it is even (using `number % 2 == 0`) and, if so, print it.

Now, let's discuss when you might want to use a stream vs. a loop:

1. Use a stream when:
   - You want to process a sequence of elements with a more functional programming style.
   - You want to chain multiple operations (e.g., filtering, mapping, reducing) on the data in a concise way.
   - You want to take advantage of lazy evaluation (i.e., computation is deferred until the result is actually needed), which can lead to performance improvements in some cases.
   - You want to perform parallel processing on the data easily. Streams provide a simple way to parallelize operations with the `parallelStream()` method.
2. Use a loop when:
   - You are working with a small number of operations or simple scenarios, where the overhead of using streams might not be justified.
   - You want to modify the underlying data structure while iterating (e.g., remove elements from a list). Note that modifying the data source while processing a stream can lead to unexpected results or concurrency issues.
   - You are working with older codebases or libraries that do not support the Stream API.
   - You need more fine-grained control over the iteration process, such as custom loop control conditions or complex loop exit strategies.

In summary, you might choose to use a stream for its functional programming style, conciseness, lazy evaluation, and easy parallelization. However, in simpler scenarios or when you need more control over the iteration, a loop can be more appropriate.

Suggestions on Model Configuration (#1)

The following was a test in seeing how ChatGPT is able to build the initial foundations of a model based on a description (this one taken from the built-in example model “Supply Chain”; available in the software from Help > Example Models). As you can see, it can give inspiration in areas ranging from the agent types to some metrics, and even the methodology itself!

However, it is important to note that the model's limited training in AnyLogic-specific contexts (due to OpenAI's training being based on publicly scraped data) may result in errors when providing specific advice on model building (e.g., mistakenly suggesting a "Decision" block). Nonetheless, it can serve as a useful starting point for identifying key industry-specific terms that warrant further exploration in a more appropriate medium for fact-finding.

I’m looking to build a supply chain model inside of AnyLogic. The following is a brief description of what I’m looking to do. Can you give me advice for the different agents I should create and anything else about how to structure the model?

A simple model of a supply chain consisting of a retailer, a wholesaler, and a factory. Customers arrive to the retailer, demand a product, and take the currently available items. The excess of demand over supply is backlogged. At the beginning of each day, the retailer and the wholesaler review their inventory levels and decide how many items to order. Similarly, the factory decides how many items to manufacture. All of them use a stationary policy. Cost is associated with ordering, manufacturing, holding products, and having a backlog of orders (shortage). The output of the simulation includes the structured mean daily cost for all supply chain elements, and the distribution of waiting times for customers to get their orders. The goal of the model is to find the inventory policy parameters for the supply chain that result in minimum cost and minimum waiting times.

To model a supply chain with a retailer, a wholesaler, and a factory in AnyLogic, you can create three agent types: Retailer, Wholesaler, and Factory. Each agent type should have its own set of variables and parameters, such as inventory levels, order quantities, and costs.
Here are some suggestions for the variables and parameters you could include in each agent type:
Retailer:

- Inventory level
- Order quantity
- Backlog of orders
- Cost per unit
- Lead time (time it takes for an order to arrive)

Wholesaler:

- Inventory level
- Order quantity from factory
- Order quantity to retailer
- Backlog of orders

- Cost per unit
- Lead time (time it takes for an order to arrive)

Factory:

- Production rate (units produced per day)
- Inventory level
- Order quantity to wholesaler
- Cost per unit
- Lead time (time it takes for an order to arrive)

To model the customers, you can use a Source block to represent the arrival process of customers at the retailer. The customers can then move through the supply chain agents to fulfill their orders.

To model the daily review of inventory levels and order quantities, you can use a Schedule block to trigger the decision-making process for each agent type. Within each agent type, you can use a Decision block to determine the order quantity based on the current inventory level and backlog of orders.

To model the costs associated with the supply chain, you can use a Cost block to track the total cost of each element, such as ordering, manufacturing, holding products, and having a backlog of orders. You can then use a Performance Measure block to calculate the mean daily cost for the entire supply chain.

To model the waiting times for customers to get their orders, you can use a Monitor block to track the time spent by each customer in the system. You can then use a Histogram block to generate the distribution of waiting times.

To find the inventory policy parameters that result in minimum cost and minimum waiting times, you can use the Optimization experiment in AnyLogic. You can define the objective function as a combination of the mean daily cost and the average waiting time, and set the optimization algorithm to search for the optimal values of the decision variables, such as order quantities and production rates.

What might be some KPIs or outputs that I should add?

Here are some additional KPIs or outputs you could add to your supply chain model:

1. Inventory turnover: This is a measure of how quickly inventory is sold and replenished. It can be calculated as the cost of goods sold divided by the average inventory level.
2. Fill rate: This is the percentage of customer orders that are fulfilled from available inventory. It can be calculated as the number of orders shipped divided by the total number of orders received.
3. Stockout rate: This is the percentage of customer orders that cannot be fulfilled due to insufficient inventory. It can be calculated as the number of backordered units divided by the

total number of units ordered.

4. Lead time: This is the time it takes for an order to be fulfilled from the time it is placed. It can be calculated as the time between the order placement and the order delivery.
5. Order cycle time: This is the time it takes to process and fulfill an order. It can be calculated as the time between the order placement and the order delivery, including any lead time.
6. Order processing cost: This is the cost of processing an order, including order entry, picking, packing, and shipping.
7. Capacity utilization: This is a measure of how effectively the supply chain is using its available capacity. It can be calculated as the actual production output divided by the maximum production capacity.

By tracking these KPIs, you can gain insights into the performance of your supply chain model and identify areas for improvement.

I'm also adding a Truck agent for transporting the product and looking to use a statechart to model its behaviors. What might be some of the states that it could be in?

If you are adding a Truck agent to transport the product, you can use a statechart to model its behaviors. Here are some states that the truck could be in:

1. Idle: This is the default state for the truck when it is not assigned to any task or mission.
2. Loading: This state represents the time when the truck is being loaded with products at the factory or warehouse.
3. En route: This state represents the time when the truck is on the road, traveling to the destination.
4. Unloading: This state represents the time when the truck is being unloaded at the retailer or wholesaler.
5. Maintenance: This state represents the time when the truck is undergoing maintenance or repair.
6. Breakdown: This state represents the time when the truck has experienced a breakdown or malfunction and is unable to perform its task.
7. Out of service: This state represents the time when the truck is out of service for any reason, such as scheduled maintenance, repairs, or inspections.

You can use transitions between these states to represent the actions that trigger the change in the truck's behavior. For example, when the truck has finished loading at the factory, it can transition to the "En route" state. Similarly, when the truck has finished unloading at the retailer or wholesaler, it can transition back to the "Idle" state. By using a statechart to model the truck's behavior, you can better understand its activities and incorporate it into your supply chain model.

What other agent types might I include in a simulation model like this?

In addition to the retailer, wholesaler, factory, and truck agents you have already mentioned, here are some other agent types that you might include in your simulation model of a supply chain:

1. Supplier: The supplier is the entity that provides raw materials or components to the factory. You could model the supplier as an agent that delivers materials to the factory on a regular basis.
2. Quality Control Inspector: This agent type could be responsible for checking the quality of the products produced by the factory, and rejecting any products that do not meet the required standards.
3. Sales Representative: This agent type could be responsible for managing the relationships with customers, and for negotiating pricing and volume agreements with retailers and wholesalers.
4. Customer Service Representative: This agent type could be responsible for responding to customer inquiries, managing returns and exchanges, and handling any other issues that arise in the customer service process.
5. Warehouse Operator: This agent type could be responsible for managing the inventory levels at the retailer or wholesaler, including receiving and storing products, picking and packing orders, and managing the shipping and logistics processes.
6. Maintenance Technician: This agent type could be responsible for maintaining and repairing the equipment used in the supply chain, including the factory machinery, the truck fleet, and the warehouse equipment.

By including these agent types in your simulation model, you can gain a more comprehensive understanding of the supply chain processes and how they interact with each other. This can help you identify bottlenecks, inefficiencies, and opportunities for improvement in the supply chain.

Some of the proposed types have some similarities in their behaviors, can this be condensed to a select list of agent types?

Yes, some of the proposed agent types may have similarities in their behaviors, and it may be possible to condense them into a select list of agent types. Here are some examples:

1. Operations Agent: This agent type could represent the entities responsible for performing the operational tasks in the supply chain, including the factory workers, warehouse operators, and truck drivers.
2. Customer Agent: This agent type could represent the entities that interact with the customers, including the sales representatives, customer service representatives, and quality control inspectors.
3. Maintenance Agent: This agent type could represent the entities responsible for maintaining and repairing the equipment in the supply chain, including the maintenance technicians and the truck mechanics.

By condensing the agent types into a select list, you can simplify the modeling process and make it easier to analyze the interactions and dependencies between the different entities in the supply chain. However, it is important to ensure that the behaviors and characteristics of the individual agents are still accurately represented within the select agent types.

What about for the whole model itself?

For the whole supply chain model, you may want to consider grouping the agents into higher-level categories based on their roles and interactions within the supply chain. Here are some potential categories:

1. Supply Chain Partner Agents: This category could include the retailer, wholesaler, and factory agents, as well as the supplier and truck agents. These agents are involved in the production and distribution of the product.
2. Customer Agents: This category could include the customers who purchase the product from the retailer.
3. Maintenance Agents: This category could include the maintenance technicians responsible for maintaining the equipment and vehicles used in the supply chain.
4. Quality Control Agents: This category could include the quality control inspectors responsible for ensuring the products meet the required standards.
5. Administrative Agents: This category could include agents responsible for managing the logistics, scheduling, and other administrative tasks associated with the supply chain, such as the scheduling of maintenance and deliveries.

By grouping the agents into these categories, you can better understand the roles and interactions of each group within the supply chain. This can help you identify opportunities for optimization and improvement, and make more informed decisions about how to manage the supply chain more effectively.

Suggestions on Model Configuration (#2)

This was another test in seeing how ChatGPT suggested modeling methodologies to use based on a given scenario. The description was inspired from the built-in example model “Epidemic and Clinic with Accumulating Concern”, a multi-method model combining agent-based and discrete event components to it (available in the software from Help > Example Models).

I’m looking to build an epidemiological simulation model in AnyLogic wherein an illness spreads through a population and people go through the stages of an illness. Some people may decide to seek out treatment after a certain point, while others may need to have a treatment intervention. Please suggest the structure of the model and what modeling methodologies I might use to model the different components.

An epidemiological simulation model in AnyLogic can be a valuable tool for understanding how an illness spreads through a population and analyzing the effectiveness of various intervention strategies. Here's a suggested structure for your model and some modeling methodologies for different components:

1. Population: Start by defining the population for your simulation. You can use an Agent-based modeling (ABM) approach for this purpose. Create an "Individual" agent with properties such as age, gender, health status, and location. Define the initial conditions for the population, like the number of susceptible, exposed, infected, and recovered individuals.
2. Disease progression: Model the stages of the illness using a Statechart within the "Individual" agent. The Statechart should contain states for different stages of the illness, such as susceptible (S), exposed (E), infected (I), and recovered (R) or removed (dead or immune). Use transitions with probabilities or time delays to represent the progression from one stage to another. You can base these probabilities on real-world data or expert opinions.
3. Transmission: Model the spread of the illness using the ABM approach. You can define interactions between individuals based on their spatial locations or social networks. The interactions can be represented by events that trigger the transmission of the disease. Consider factors such as the probability of transmission, contact rates, and the duration of infectiousness.
4. Treatment-seeking behavior: Add a decision-making process within the "Individual" agent for seeking treatment. This can be modeled using a combination of decision trees or rule-based systems. Factors influencing treatment-seeking behavior can include the severity of symptoms, access to healthcare, cost, and cultural factors. You can also incorporate the impact of treatment on the progression of the illness and the individual's recovery.
5. Treatment intervention: Model the intervention process using a discrete-event simulation (DES) approach. Define a "Healthcare Facility" agent that can accept patients, provide treatment, and manage limited resources like hospital beds, ventilators, or staff. You can use queues and resource allocation strategies to manage the patient flow and treatment process. Incorporate intervention policies such as mass vaccination campaigns or targeted treatments for specific subpopulations.
6. Analysis and visualization: Implement data collection and visualization tools to analyze the dynamics of the illness spread and the effectiveness of different intervention strategies. Use

AnyLogic's built-in tools like time plots, histograms, and GIS maps for visualization. Perform sensitivity analysis to understand the impact of various parameters on the outcomes of the simulation.

In summary, use a combination of Agent-based modeling (ABM) for representing the population and disease transmission, Statecharts for modeling the stages of the illness, decision trees or rule-based systems for treatment-seeking behavior, and discrete-event simulation (DES) for modeling the treatment intervention process. This hybrid approach will help you capture the complexities of the epidemiological system and enable you to explore various scenarios and intervention strategies.

Code Explanation: Supply Chain

The following is based on the Supply Chain model built as part of AnyLogic training (the closest analogous model in the built-in examples is Product Delivery; available in the software from Help > Example Models). One step in the training, when building the "orderProducts" event in the Retailer agent, has users writing code. As this is a common source of confusion to those new to programming, we wanted to see how ChatGPT could explain it – even without any real context on the model itself!

We also explore how it responds to being asked to rewrite code that includes functions specific to AnyLogic contexts (e.g., "getNearestAgentByRoute"). As can be seen, it makes some incorrect assumptions which lead to errors inside of AnyLogic when the instructions are blindly followed. However this also shows its ability to help with learning about the errors and can help with finding a solution.

I'm working on a model inside the AnyLogic software (uses Java) and encountered the following code, can you explain it to me?

```
Order order = new Order();
order.amount = uniform_discr(5, 20);
order.retailer = this;
Truck truck = getNearestAgentByRoute(filter(main.trucks, t -
t.inState(Truck.AtDistributor)));
if (truck != null) { send(order, truck); }
```

Certainly! Let me break down this code snippet for you:

1. `Order order = new Order();` - This creates a new instance of the `Order` class, which is presumably a class that has been defined elsewhere in the codebase. The `new` keyword creates a new object of that class and assigns it to the variable `order`.
2. `order.amount = uniform_discr(5, 20);` - This sets the `amount` property of the `order` object to a random integer between 5 and 20 (inclusive) generated by the `uniform_discr` method. Presumably, `uniform_discr` is a method that has been defined elsewhere in the codebase to generate a discrete uniform distribution.
3. `order.retailer = this;` - This sets the `retailer` property of the `order` object to the current object instance, which is presumably an instance of a class that has a `retailer` property.
4. `Truck truck = getNearestAgentByRoute(filter(main.trucks, t - t.inState(Truck.AtDistributor)));` - This finds the nearest `Truck` object that is in the `AtDistributor` state, using the `getNearestAgentByRoute` method. Presumably, `getNearestAgentByRoute` is a method that has been defined elsewhere in the codebase to find the nearest object of a given type by following a specified route. The `filter` method is used to filter the `main.trucks` collection to only include trucks that are currently in the `AtDistributor` state.
5. `if (truck != null) { send(order, truck); }` - This checks if a `Truck` object was found in the previous step. If so, it sends the `order` object to that truck using the `send` method. Presumably, `send` is a method that has been defined elsewhere in the codebase to send an object from one agent to another.

Overall, this code snippet seems to be generating an `Order` object with a random amount and

assigning the current object instance to its `retailer` property. It then finds the nearest `Truck` object that is in the `AtDistributor` state and sends the `Order` object to that truck if one is found.

What does it mean for something to be “null”?

In programming, the value `null` typically represents the absence of a value or a lack of an object. It is a special value that can be assigned to any reference type in many programming languages, including Java, C++, and Python.

When a variable is assigned the value `null`, it means that it does not refer to any object in memory. This can happen in several ways, such as when a variable is declared but not initialized, or when an object that the variable previously referred to has been deleted or gone out of scope.

When code attempts to use a `null` reference, such as by calling a method or accessing a property on a `null` object, it will typically result in a runtime error, such as a `NullPointerException` in Java. Therefore, it is important to check whether a reference is `null` before using it in code, using conditional statements or other methods, to avoid such errors.

In summary, `null` is a special value in programming that represents the absence of a value or object, and its use should be handled with care to avoid runtime errors.

I’m not a good programmer and won’t remember this. Can you add some comments to my code?

Sure! Here is an example of how you could add comments to your code snippet to make it more understandable:

```
// Create a new instance of the Order class
Order order = new Order();

// Set the amount property of the order object to a random integer between 5 and
20
order.amount = uniform_discr(5, 20);

// Set the retailer property of the order object to the current object instance
order.retailer = this;

// Find the nearest truck that is currently at the distributor
Truck truck = getNearestAgentByRoute(filter(main.trucks, t -
t.inState(Truck.AtDistributor)));

// If a truck was found, send the order object to it
if (truck != null) {
    send(order, truck);
```

```
}
```

In these comments, each line of code is briefly explained in plain English. The comments should help you and others better understand what the code does without having to read and interpret the code itself.

Rather than the custom filter function, can you rewrite to use a loop instead?

Sure, here's an example of how you could rewrite the code to use a loop instead of the custom `filter` function:

```
// Find the nearest truck that is currently at the distributor
Truck truck = null;
double shortestDistance = Double.POSITIVE_INFINITY;

for (Truck t : main.trucks) {
    if (t.inState(Truck.AtDistributor)) {
        double distance = getDistance(this, t);
        if (distance < shortestDistance) {
            shortestDistance = distance;
            truck = t;
        }
    }
}

// If a truck was found, send the order object to it
if (truck != null) {
    send(order, truck);
}
```

In this example, we use a `for` loop to iterate through each `Truck` object in `main.trucks`. We check if the truck is currently in the `AtDistributor` state using the `inState` method, and if so, we calculate the distance between the `this` object (which presumably represents the current retailer) and the truck using a hypothetical `getDistance` method. We then update the `shortestDistance` and `truck` variables if the current truck is closer than the previous closest truck.

After the loop completes, if a truck was found (`truck` is not `null`), we send the `order` object to it using the `send` method.

Note that the exact implementation of the `getDistance` method will depend on the specifics of your code and what type of distance metric you want to use. This is just a placeholder method for demonstration purposes.

When I try this, I get the following error:
Description: The method getDistance(double, double, double, double) in the type Utilities is not applicable for the arguments (Retailer, Truck). Location: Supply Chain/Retailer/orderProducts - Event

Based on the error message you provided, it seems like the `getDistance` method is expecting four `double` arguments, but you are passing in a `Retailer` and `Truck` object instead.

To fix this, you need to modify the `getDistance` method to accept `Retailer` and `Truck` objects as arguments, and extract the relevant coordinates from these objects to calculate the distance between them. Here's an updated example of how you could implement the `getDistance` method:

```
public double getDistance(Retailer retailer, Truck truck) {
    double x1 = retailer.x; // assuming x is a property of the Retailer class
    double y1 = retailer.y; // assuming y is a property of the Retailer class
    double x2 = truck.x; // assuming x is a property of the Truck class
    double y2 = truck.y; // assuming y is a property of the Truck class
    return Math.sqrt(Math.pow(x2 - x1, 2) + Math.pow(y2 - y1, 2));
}
```

In this example, we assume that the `Retailer` and `Truck` classes both have `x` and `y` properties that represent their respective coordinates. We use these properties to calculate the Euclidean distance between the two points using the Pythagorean theorem.

With this updated implementation of `getDistance`, you should be able to use the loop-based approach to find the nearest truck without encountering the previous error.

Yeah it seems like you made up the `getDistance` function with the two arguments! However I just looked it up and there is a built-in function called `getDistance` and it expects x1, y1, x2, y2 (for my future reference: https://anylogic.help/api/com/anylogic/engine/Utilities.html#getdistance).
You can get the 'x' and 'y' of any agent (Retailer or Truck) with `getX()` and `getY()` (also for my future reference: https://anylogic.help/anylogic/agentbased/agent-movement.html#functions).

I apologize for the confusion earlier. Thank you for letting me know about the built-in `getDistance`, `getX`, and `getY` functions in AnyLogic.

With this in mind, you can modify the loop-based approach to use the `getDistance` function as follows:

```
// Find the nearest truck that is currently at the distributor
Truck truck = null;
double shortestDistance = Double.POSITIVE_INFINITY;

for (Truck t : main.trucks) {
    if (t.inState(Truck.AtDistributor)) {
        double distance = getDistance(getX(), getY(), t.getX(), t.getY());
        if (distance < shortestDistance) {
            shortestDistance = distance;
            truck = t;
        }
    }
}
```

```
// If a truck was found, send the order object to it
if (truck != null) {
    send(order, truck);
}
```

In this updated code, we iterate through each `Truck` object in `main.trucks`, and check if the truck is currently in the `AtDistributor` state using the `inState` method. If the truck is at the distributor, we use the built-in `getDistance` function to calculate the Euclidean distance between the retailer (using `getX()` and `getY()` methods) and the truck (using `t.getX()` and `t.getY()` methods).

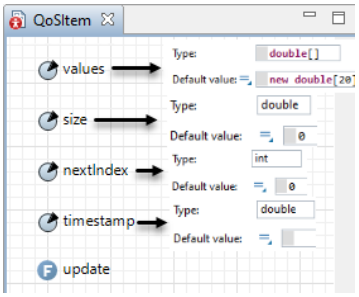
If the current truck is closer than the previous closest truck, we update the `shortestDistance` and `truck` variables accordingly. Finally, if a truck was found (`truck` is not `null`), we send the `order` object to it using the `send` method.

I hope this updated code resolves the issue you were experiencing earlier.

Code Explanation: Adaptive Supply Chain

In looking for good use-cases for having ChatGPT explain code, one example of potentially confusing code can be seen in the "QoSItem" agent of the built-in example model "Adaptive Supply Chain" (found in the software from Help > Example Models).

In this agent, there's a function called "update" which makes some changes to 4 of its parameters. Without any further context, it can be hard to deduce the reasoning for it. As ChatGPT has been trained on an immense amount of data, it can easily detect common patterns used in coding which it can help teach you about.



Explain what the following java code from a function that takes "value" and "time" (two numbers) as input and from an object where "values" is a list of numbers:

```
values[nextIndex] = value;
size = min( size + 1, values.length );
nextIndex = ( nextIndex + 1 ) % values.length;
timestamp = time;
```

This Java code updates the state of an object that maintains a circular buffer of numerical values. Here's a step-by-step breakdown of what each line does:

1. `values[nextIndex] = value;` : This sets the value at the index `nextIndex` of the `values` list to the `value` input parameter. This effectively overwrites the oldest value in the buffer with the newest value.
2. `size = min( size + 1, values.length );` : This updates the `size` variable to be the minimum of `size + 1` and the length of the `values` list. This ensures that the `size` variable tracks the number of elements in the buffer, but never exceeds the maximum capacity of the buffer (which is the length of the `values` list).
3. `nextIndex = ( nextIndex + 1 ) % values.length;` : This updates the `nextIndex` variable to be the next index in the circular buffer. This is done by incrementing `nextIndex` by 1 and then using the modulo operator (%) to wrap around to the beginning of the buffer when `nextIndex` reaches the end.
4. `timestamp = time;` : This sets the `timestamp` variable to the `time` input parameter. This is used to associate a timestamp with the most recently added value in the buffer.

Can you explain in simple, non-technical terms, what a circular buffer is, and why it might be useful?

Sure! A circular buffer (also called a circular queue or ring buffer) is a data structure that is used to efficiently manage a fixed-size buffer of items. The buffer is organized as a ring, meaning that the first and last items are adjacent and the buffer can be treated as a continuous loop.

When new items are added to the buffer, they overwrite the oldest items in a circular fashion. This means that the buffer always contains the most recent n items (where n is the size of the buffer) and discards the oldest items automatically.

Circular buffers are useful in situations where you need to efficiently manage a fixed-size buffer of items that are produced and consumed at different rates. For example, in computer graphics, a circular buffer might be used to store the most recent frames of a video stream, so that the most recent frames can be displayed quickly without having to buffer the entire video. In embedded systems, circular buffers are often used to manage input and output data streams between different components of the system.

This is found in a simulation model depicting a supply chain, in an agent representing a qos item. What might the reason this type of buffer is needed?

In a simulation model depicting a supply chain, the circular buffer might be used to represent a queue of items that are waiting to be processed or shipped. The circular buffer allows the model to track the most recent n items that are waiting in the queue, and discards the oldest items automatically when new items arrive.

For an agent representing a QoS (Quality of Service) item, the circular buffer might be used to track the performance of the item over time. For example, the circular buffer might store the response times of the QoS item for the most recent n requests. This allows the model to track the performance of the QoS item over time, and to discard outdated performance data automatically.

The circular buffer is useful in these situations because it allows the model to efficiently manage a fixed-size buffer of items, without having to allocate memory dynamically or manage the buffer manually. The circular buffer is also easy to implement and understand, which makes it a popular choice for simulation models.

Database View Creation

This scenario was to see, based on the structure of a database, if ChatGPT could effectively construct a HyperSQL query to consolidate the tables in a format conducive for being the input to a population. Without this, the alternative would require us to pre-process the information before importing it into the software or heavily consulting the HyperSQL manual for how to construct queries (as the functions available to it have differences compared to other more common SQL-flavors).

I'm looking to request some database queries. First, let me tell you about my 4 database tables:

1. "location": location_id(int), latitude(double), longitude(double)
2. "employee": employee_id(int), first_name(string), last_name(string), employed_at(int), position(string)
3. "factory" : location_id(int), name(string)
4. "retailer": location_id(int), name(string), request_from(string), request_amount(float)

Here are the constraints:

- In "location", location_id is primary key
- In "employee", employed_at is a foreign key to location.location_id
- In "factory", location_id is a foreign key to location.location_id and name is a primary key
- In "retailer", location_id is a foreign key to location.location_id and request_from is a foreign key to factory.name

Thank you for providing the information about your database tables and their constraints. Now, how can I help you?

For HyperSQL, write a query that will display the following information about each factory:

- Its name
- Its latitude and longitude
- The number of employees for each of the three types of positions: Receiving, Production, and Manager
- The total 'request_amount' for the retailers who are requesting from it

Here's a HyperSQL query that retrieves the requested information:

```
SELECT f.name,
       l.latitude,
       l.longitude,
       SUM(CASE WHEN e.position = 'Receiving' THEN 1 ELSE 0 END) AS receiving_count,
       SUM(CASE WHEN e.position = 'Production' THEN 1 ELSE 0 END) AS production_count,
       SUM(CASE WHEN e.position = 'Manager' THEN 1 ELSE 0 END) AS manager_count,
       COALESCE(SUM(r.request_amount), 0) AS total_request_amount
FROM factory f
```

```
JOIN location l ON f.location_id = l.location_id
LEFT JOIN employee e ON f.location_id = e.employed_at
LEFT JOIN retailer r ON f.name = r.request_from
GROUP BY f.name, l.latitude, l.longitude;
```

The numbers looked too large, so I looked through the data and turns out the counts are all some multiple of the actual count….and it's different for each location (e.g., 3x or 4x). Do you know why?

I apologize for the confusion. The issue is likely caused by the multiple LEFT JOIN operations, which can result in duplicate entries when there are multiple matching records in both the "employee" and "retailer" tables. To address this issue, we can use subqueries to calculate the employee counts and total request amounts separately before combining the results. Here's an updated query:

```
SELECT f.name,
       l.latitude,
       l.longitude,
       COALESCE(emp.receiving_count, 0) AS receiving_count,
       COALESCE(emp.production_count, 0) AS production_count,
       COALESCE(emp.manager_count, 0) AS manager_count,
       COALESCE(req.total_request_amount, 0) AS total_request_amount
FROM factory f
JOIN location l ON f.location_id = l.location_id
LEFT JOIN (
    SELECT employed_at,
           COUNT(CASE WHEN position = 'Receiving' THEN 1 END) AS receiving_count,
           COUNT(CASE WHEN position = 'Production' THEN 1 END) AS production_count,
           COUNT(CASE WHEN position = 'Manager' THEN 1 END) AS manager_count
    FROM employee
    GROUP BY employed_at
) emp ON f.location_id = emp.employed_at
LEFT JOIN (
    SELECT request_from,
           SUM(request_amount) AS total_request_amount
    FROM retailer
    GROUP BY request_from
) req ON f.name = req.request_from;
```

In this updated query, we calculate employee counts for each position in the first subquery (emp) and total request amounts in the second subquery (req) before joining them to the main query. This should prevent the multiplication issue and provide the correct counts for each factory.

Code for Agent Placement and Visibility

This example explores customizing the visual aspects and positioning of agents in a model. The log below is for an expanded version of what was shown in the webinar.

For context on the model, it's intended to be the start of a supply chain scenario. However, rather than having 3 different agent types for each "tier" (e.g., Manufacturer, Warehouse, Customer), there's instead one agent type ("Location") with a parameter to distinguish which sub-type it is. This modeling strategy is applicable when the agent types in question are similar enough in their implementation to be a valid case for consolidation, which might not be applicable in actual supply chain models.

The other concept to note is that we have 3 animation icons stacked on one another, with the idea that only one will be visible depending on the parameter (similar to how the "SolarPanel" agent type works in the example model "Solar Panel Production Line", found in the software from Help > Example Models).

The initial implementation had 1 population to test the visibility (both for the 3 valid values and 1 invalid value) then 3 populations were added for each sub-type. Screenshots of the model in both stages can be found after the chat log.

I'm using a Java-based simulation software. In one of my agent (class) types, there are three shapes with a "Visible" code field that expects to evaluate to true/false. There's a parameter called "tier" (int). I need to know what Java code to write for each of the 3 shapes so that they're visible if the tier equals 1, 2, or 3 (respectively).

You can use a simple equality check to compare the "tier" value with the desired value for each shape. Assuming that the "Visible" code field should return a boolean value, you can write the following Java code for each of the 3 shapes:

For Shape 1 (visible when tier equals 1):

```
tier == 1
```

For Shape 2 (visible when tier equals 2):

```
tier == 2
```

For Shape 3 (visible when tier equals 3):

```
tier == 3
```

Just replace the existing code in the "Visible" code field of each shape with the appropriate code snippet from above.

I'm creating a population of these objects and want to test the visibility conditions. For a field that assigns the tier value, there's a variable called "index" (int). What code do I need so it cycles between tiers 1 to 4?

If you want to cycle between tiers 1 and 4 using the "index" variable, you can use the modulo operator. Since there are 4 tiers in this range (1, 2, 3, 4), you will take the remainder of the division of "index" by 4 and then add 1 to the result. Here's the code you need:

```
tier = (index % 4) + 1;
```

This will assign the values 1, 2, 3, 4 to the “tier” variable in a cyclical manner as the “index” variable increases.

I’m now trying to set the X position field (double) of each object between 100 and 900, spread out equally, and centered at the middle (500). Again, you have access to “index” and can also get the total number of objects via “self.getPopulation().size()”

To set the X position of each object between 100 and 900, spread out equally and centered at the middle (500), you can calculate the spacing between the objects and then use the “index” variable to determine the position of each object. Here’s the code you need:

```
double startPosition = 100.0;
double endPosition = 900.0;
int totalObjects = self.getPopulation().size();

double spacing = (endPosition - startPosition) / (totalObjects - 1);
double xPos = startPosition + (index * spacing);

// Centering at the middle (500)
double center = (startPosition + endPosition) / 2.0;
xPos -= center;
xPos += 500;
```

This code calculates the spacing between objects based on the total number of objects and sets the X position of each object using the “index” variable. It then adjusts the position to be centered at 500.

The field I’m entering the code into expects a single value. Can you condense this to a single line that evaluates to the correct number?

Sure, you can condense the code into a single line that calculates the X position directly. Here’s the single-line code you need:

```
100 + (index * ((900.0 - 100.0) / (self.getPopulation().size() - 1))) - (((100.0 + 900.0) / 2.0) - 500)
```

This code calculates the X position for each object using the “index” variable and centers it around 500, as required.

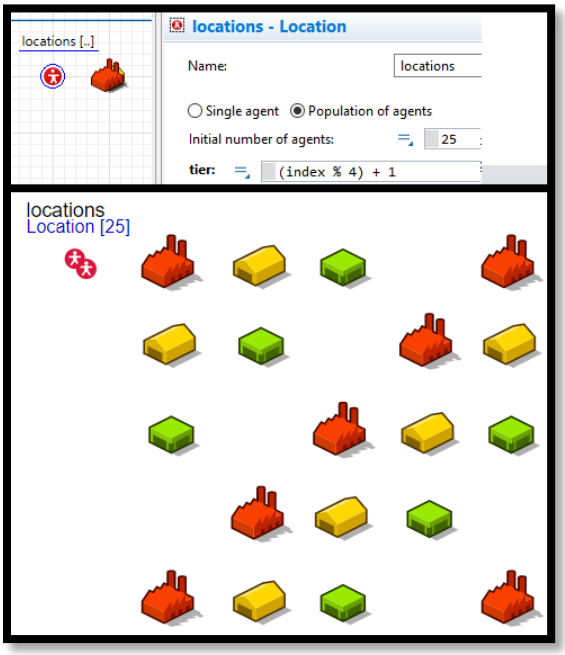
This is great but it doesn’t handle the case when there’s only one object in the group (it causes the X to be NaN due to divide by 0)

Apologies for the oversight. To handle the case when there’s only one object in the group, you can use a ternary conditional operator to check if the total number of objects is greater than 1. If it is, then calculate the X position as before; otherwise, set the X position directly to 500. Here’s the revised single-line code:

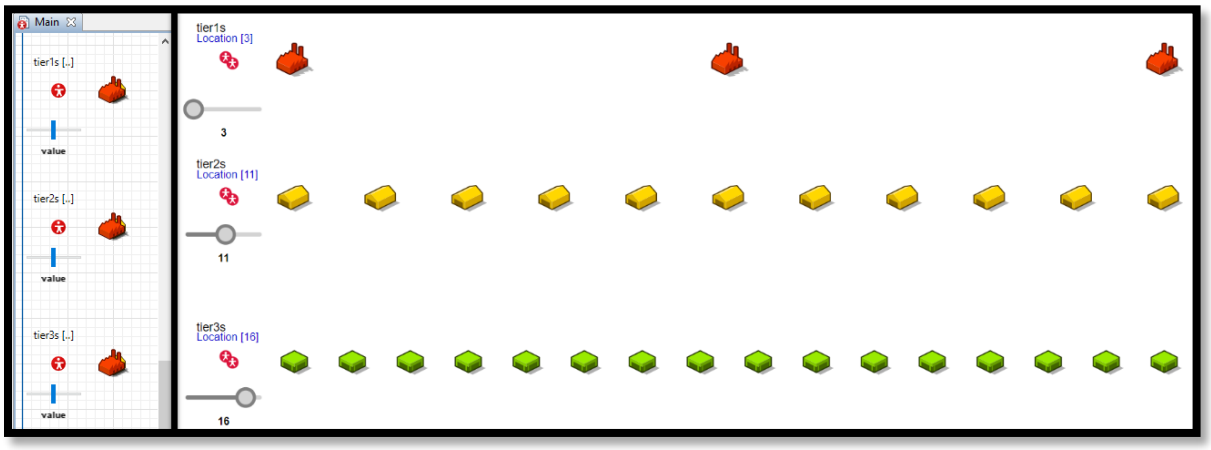
```
(self.getPopulation().size() > 1) ? (100 + (index * ((900.0 - 100.0) /
```

```
(self.getPopulation().size() - 1))) - (((100.0 + 900.0) / 2.0) - 500)) : 500
```

This code handles the case when there's only one object in the group and sets the X position to 500, avoiding any division by zero issues.



Results from testing initial icon visibility (in "Arranged" network layout)



Results from including 3 populations and position equations

Recreating Bass Diffusion Model

We wanted to see if ChatGPT could combine the information it was trained on regarding what System Dynamics is and the established formulation of the Bass diffusion model.

While it could be argued that it's simply regurgitating information, what's interesting to consider is that it can provide us with example values and that it seems to "understand" (at least on some level) how the dynamics of the system might play out. Keep in mind that, as a language model, it can't directly perform calculations; rather it seems to infer what happens based on what it "knows" about the role of different math operators.

I'm looking to build a System Dynamics simulation inside of AnyLogic depicting a bass innovation diffusion model (innovation diffusion as driven by advertising and word of mouth feedback). What objects will I need and what should they be set to?

To build a System Dynamics simulation of a Bass innovation diffusion model in AnyLogic, you'll need to set up several objects and variables. Here's a step-by-step guide:

1. Create a new System Dynamics model in AnyLogic.
2. Add the following stocks:
    a. Potential Adopters (P): Represents the population that has not yet adopted the innovation.
    b. Adopters (A): Represents the population that has adopted the innovation.
3. Set the initial values for the stocks:
    a. Potential Adopters (P): Set the initial value to the total potential market size.
    b. Adopters (A): Set the initial value to the number of initial adopters, if any.
4. Add the following flows:
    a. Adoption Rate: Represents the rate at which potential adopters adopt the innovation.
5. Add the following variables and set their values:
    a. Coefficient of Innovation (p): Represents the influence of advertising on adoption. Set it to the corresponding value for your model.
    b. Coefficient of Imitation (q): Represents the influence of word of mouth on adoption. Set it to the corresponding value for your model.
    c. Total Population (M): Represents the total potential market size. Set it to the corresponding value for your model.
6. Define the flow equation for Adoption Rate: Adoption Rate = (P * (p + q * (A / M)))
7. Connect the objects:
    a. Connect Adoption Rate to both Potential Adopters and Adopters.
        - Adoption Rate should decrease the value of Potential Adopters.

- Adoption Rate should increase the value of Adopters.

8. Run the simulation to observe the behavior of the model over time. You can use plots and charts to visualize the adoption process.

Remember to adjust the model parameters (p, q, and M) based on the specific case you're simulating, and to validate your model using historical data if available.

What are some sample initial values to test?

To test your Bass innovation diffusion model, you can use some hypothetical initial values. Here's a sample set of initial values you can use to test the model:

1. Total Population (M): 10,000
   - This represents the total potential market size.
2. Initial Potential Adopters (P): 9,990
   - This represents the population that has not yet adopted the innovation at the beginning of the simulation.
3. Initial Adopters (A): 10
   - This represents the number of initial adopters at the beginning of the simulation.
4. Coefficient of Innovation (p): 0.01
   - This represents the influence of advertising on adoption. A value of 0.01 means that 1% of potential adopters are influenced by advertising to adopt the innovation.
5. Coefficient of Imitation (q): 0.4
   - This represents the influence of word of mouth on adoption. A value of 0.4 means that 40% of potential adopters are influenced by word of mouth to adopt the innovation.

Please note that these initial values are for illustration purposes only. You will need to adjust the parameters based on the specific case you're simulating, and it's essential to validate your model using historical data or empirical evidence if available.

With these initial values, what do you expect to happen to the values when the model is executed?

Given the hypothetical initial values for the Bass innovation diffusion model, we can expect the following behavior when the model is executed:

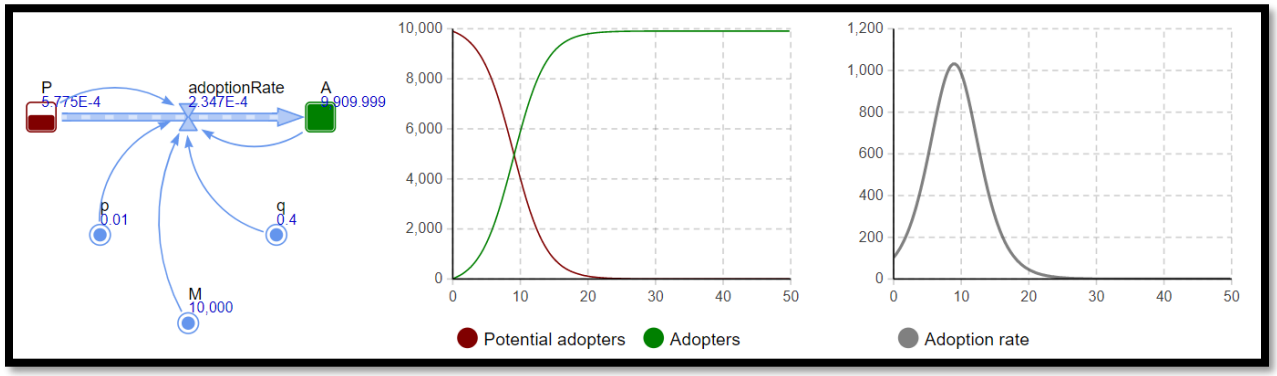
1. Potential Adopters (P) will decrease over time as more people adopt the innovation.
2. Adopters (A) will increase over time as more people adopt the innovation.
3. The adoption rate will initially be relatively low, driven primarily by the Coefficient of Innovation

(p) and the small number of initial adopters.

4. As the number of adopters (A) increases, word of mouth will play a more significant role in the adoption process, leading to a higher adoption rate. This will be reflected in the increasing value of (q * (A / M)) in the Adoption Rate equation.
5. The adoption rate will reach a peak and then start to decline as the pool of potential adopters decreases.
6. Eventually, the adoption rate will approach zero, and the number of adopters (A) will be close to the total potential market size (M). The number of potential adopters (P) will also approach zero.

The diffusion process will follow an S-shaped curve, which is a typical characteristic of innovation diffusion models. The adoption rate will show a bell-shaped curve, indicating the spread of the innovation over time.

Keep in mind that these are general expectations based on the provided initial values, and the actual results may vary depending on the specific parameters and assumptions used in your model. Always validate your model using historical data or empirical evidence if available.



Results from implementing its instructions

Creating novel System Dynamics Model

It's reasonable to assume that ChatGPT had enough training examples of System Dynamics modeling and the Bass Diffusion problem – both widely discussed topics – to be able to recreate it effectively by essentially regurgitating the information.

To see how it would perform on a more novel scenario, we found a paper on using System Dynamics to model burnout experienced by individuals: "A Proof-of-Concept System Dynamics Simulation Model of the Development of Burnout and Recovery Using Retrospective Case Data" by Veldhuis et al. (https://www.ncbi.nlm.nih.gov/pmc/articles/PMC7459661/).

The initial idea was to recreate a simplified version of the stock and flow shown in Figure 2 of the paper, using some proxy variables to stand in for some of the variables shown in the paper. An implementation was never tested, but the results were still interesting.

I'm looking to build a system dynamics model in AnyLogic representing interest level and endurance for an employee's work based on their capacity for effort, the desirability of work, and skill level. I plan to have a toggleable variable called "isWorking" (0=not working, 1=working). Describe the objects and connections I'll need to implement this.

To build a system dynamics model in AnyLogic representing an employee's interest level and endurance based on their capacity for effort, desirability of work, and skill level, you can follow these steps:

1. Create Stock objects:
   - Interest Level: Represents the employee's current interest level in their work.
   - Endurance: Represents the employee's current ability to maintain their effort over time.
2. Create Variable objects:
   - Capacity for Effort: Represents the employee's ability to exert effort in their work.
   - Desirability of Work: Represents how attractive or engaging the work is for the employee.
   - Skill Level: Represents the employee's competency in their work.
   - isWorking: A toggleable variable (0=not working, 1=working) to determine the employee's work status.
3. Create Flow objects:
   - Interest_Inflow: Represents factors that increase the employee's interest level, such as the desirability of work, skill level, and capacity for effort.
   - Interest_Outflow: Represents factors that decrease the employee's interest level, such as burnout or disengagement when not working.
   - Endurance_Inflow: Represents factors that increase the employee's endurance, such as the capacity for effort, skill level, and interest level.
   - Endurance_Outflow: Represents factors that decrease the employee's endurance, such as exhaustion or disengagement when not working.
4. Define equations for Flows:
   - Interest_Inflow: Interest_Inflow = isWorking * (Desirability of Work + Skill Level) * Capacity for Effort
   - Interest_Outflow: Interest_Outflow = (1 - isWorking) * Interest Level / Time Constant

- Endurance_Inflow: Endurance_Inflow = isWorking * (Capacity for Effort + Skill Level) * Interest Level
- Endurance_Outflow: Endurance_Outflow = (1 - isWorking) * Endurance / Time Constant

5. Connect Flows to Stocks:
   - Connect Interest_Inflow and Interest_Outflow to the Interest Level stock.
   - Connect Endurance_Inflow and Endurance_Outflow to the Endurance stock.
6. Set initial values for Stocks and Variables:
   - Define initial values for Interest Level, Endurance, Capacity for Effort, Desirability of Work, and Skill Level based on your specific use case.
   - Set the initial value for isWorking to 0 or 1 depending on the employee's initial work status.
7. Add a Time Constant:
   - Add a time constant to represent the time it takes for interest and endurance to decay when not working. Adjust the value based on your specific use case.
8. Run the simulation:
   - Set up the simulation parameters, including the time unit and the time horizon.
   - Run the simulation and observe the changes in Interest Level and Endurance as you toggle the isWorking variable.

By implementing these objects and connections, you can create a system dynamics model in AnyLogic that represents an employee's interest level and endurance based on their capacity for effort, desirability of work, and skill level.

AnyLogic-Specific Prompt Engineering

This section explores the idea of having an easy to copy/paste initial prompt that gives ChatGPT enough context on AnyLogic to provide more useful help in subsequent queries.

The first two chats, not included in the webinar, use an initial prompt that was used to write a function for the "selectOutput5" block (with the "Exit number" option) in built-in Job Shop model (from Help > Example Models).

The third chat includes a revised version of the prompt, which is what was shown in the webinar, in a scenario for writing a function for a custom model.

Prompt #1, Attempt #1

The generic prompt is in the first cell below with the request in the following message. It was interrupted mid-way through its generation because of how widely off the mark it was.

I'm using the AnyLogic simulation software and need help coding. Things to know first:

- The programming language is Java.
- users create Agent types that can be considered Java classes.
- Objects, such as parameters (like constructor arguments), variables, collections, functions, and flowchart blocks, are dragged in to the visual canvas for an Agent and edited in a properties panel.
- There are action callbacks in the properties of Agent types and some objects, such as the "On startup" (which acts similar to the "main" function in a traditional Java application). Flowchart blocks have callbacks for actions relevant to their function (e.g., on enter, on exit, on seize, on delay, etc.).
- Flowchart blocks are parameterized with a certain Agent type. In their action callbacks, there is typically a local variable "self" to refer to the block and another one referring to the current agent passing through it - typically it's "agent" but it can also be "ped" or "car" if using a block from a different library (Pedestrian or Road Traffic respectively).
- In an Agent's properties, there's a dedicated imports section where any Java imports go.

With that, I'll tell you about my model and you should give advice on how to implement a solution using this information.

Sure, please go ahead and explain your model, and I'll do my best to help you with the implementation.

In my model I have three CNCMachine agents (named cncMachineA, cncMachineB, and cncMachineC). I need a function that will return a number from 1 to 3, corresponding to each agent, returning the first number for the machine that isn't currently "blocked" – this can be checked by calling a "isBlocked" function of the object named "onePalletAtMachine" which is inside the CNCMachine agent type

Okay, here's an example function that you can use to accomplish what you need:

```
import java.util.Random;

public int getAvailableMachine() {
    int[] machines = {1, 2, 3}; // Array of machine numbers
    Random rand = new Random(); // Create a new random number generator
    int index = rand.nextInt(machines.length); // Generate a random index into the
array
    for (int i = 0; i < machines.length; i++) { // Loop through the array,
starting at the random index
        int machine = machines[(index + i) % machines.length]; // Get the machine
number at the current index
        CNCMachine cncMachine = (CNCMachine) getAgent("cncMachine" + machine); //
Get the CNCMachine agent object for the current machine
        if (!cnc
```

The last prompt was intentionally interrupted.

Prompt #1, Attempt #2

The prompt and the request made are the exact same as in the GPT3.5 version, but this is using GPT4. As it can be seen, the output was vastly more improved than the first attempt.

I'm using the AnyLogic simulation software and need help coding. Things to know first:

- The programming language is Java.
- users create Agent types that can be considered Java classes.
- Objects, such as parameters (like constructor arguments), variables, collections, functions, and flowchart blocks, are dragged in to the visual canvas for an Agent and edited in a properties panel.
- There are action callbacks in the properties of Agent types and some objects, such as the "On startup" (which acts similar to the "main" function in a traditional Java application). Flowchart blocks have callbacks for actions relevant to their function (e.g., on enter, on exit, on seize, on delay, etc.).
- Flowchart blocks are parameterized with a certain Agent type. In their action callbacks, there is typically a local variable "self" to refer to the block and another one referring to the current agent passing through it - typically it's "agent" but it can also be "ped" or "car" if using a block from a different library (Pedestrian or Road Traffic respectively).
- In an Agent's properties, there's a dedicated imports section where any Java imports go.

With that, I'll tell you about my model and you should give advice on how to implement a solution using this information.

It seems like you have a basic understanding of how AnyLogic works. Now, I'll need some information about your model, the problem you're trying to solve, and any specific requirements or constraints. Once you provide this information, I'll be able to guide you on how to implement a solution using AnyLogic's features and the Java programming language.

In my model I have three CNCMachine agents (named cncMachineA, cncMachineB, and cncMachineC). I need a function that will return a number from 1 to 3, returning the first one that isn't currently blocked. This can be checked by calling a "isBlocked" function of the object named "onePalletAtMachine" which is inside the CNCMachine agent type.

To implement this solution in your AnyLogic model, you can follow these steps:

1. Create a function within your main agent (or any suitable agent where you need this functionality) that returns an integer. You can name this function `getFirstUnblockedMachine`.
2. In the function code, you need to access the `isBlocked()` function of the `onePalletAtMachine` object inside each CNCMachine agent. You can use the `getAgent()` function to access these objects. If a machine is not blocked, return the respective number (1, 2, or 3).

Here is a possible implementation for the `getFirstUnblockedMachine` function:

```
int getFirstUnblockedMachine() {
    if (!cncMachineA.onePalletAtMachine.isBlocked()) {
        return 1;
    } else if (!cncMachineB.onePalletAtMachine.isBlocked()) {
        return 2;
    } else if (!cncMachineC.onePalletAtMachine.isBlocked()) {
        return 3;
    } else {
        // All machines are blocked; return -1 or any suitable value to indicate
this situation.
        return -1;
    }
}
```

Now, you can call this function whenever you need to find the first unblocked machine. The function will return the number of the first unblocked machine or -1 if all machines are blocked.
Note that this implementation assumes that the main agent (or the agent where this function is placed) has direct access to the `cncMachineA`, `cncMachineB`, and `cncMachineC` agents. If these agents are not directly accessible, you might need to adjust the code to reference them correctly.

Prompt #2

The following is a modified version of the original prompt shown in the previous examples which is written with agent-based models in mind (it wasn't found to be better or worse).

I'm building a simulation model in AnyLogic and need help. But first some rules about how Java programming in AnyLogic works:

- Creating a new Agent type is analogous to creating a new Java class. By default, there is a Main agent type already created in the model.
- Objects for Java programming (e.g., functions, variables, parameters, collections) and for simulation building (e.g., discrete event flowcharts) are dragged in from the Palette panel to the Agent type and configured in the Properties panel
- Class constructors should be described as adding a Parameter object
- Any Java imports are to be added to a dedicated "Imports" section in the Agent type's properties
- A "population" of an agent type is analogous to an ArrayList of that type with extra features. Notably: AnyLogic automatically generates a function for a population called "add_X" where X is the name of the population and the arguments are the parameter values for the new agent, which gets created and added when called.

Great, thanks for the rules and context! Now, how can I help you with your AnyLogic simulation model? Please let me know what you're trying to model or any specific questions you have about Java programming in AnyLogic.

In my model, I have an Agent type called "State" which has parameters of "state" (String) and "population" (int). In my Main agent, I have a parameter called "year" (int) and an empty population called "states". I want a function in Main with the following specifications:

- Takes as input a number called "year"
- Performs a basic HTTP request to the following URL, replacing "INPUT" with the given year, unless the year is 0, in which it should pass "latest": https://datausa.io/api/data?drilldowns=State&measures=Population&year=INPUT
- Parses the output (JSON) using the Jackson library. The returned object will have a key called "data", mapping to a list of objects. But if the list is empty, it should call the "error" function and pass a message alerting the user. Otherwise, it'll loop over the data array and create add a new State agent to the population, passing in the JSON object's "State" and "Population" keys.
- The function has no return

To implement the function you described, follow these steps:

1. Add the required imports for the Jackson library and HTTP requests in the "Imports" section of

your Main agent type's properties:

```
import java.net.HttpURLConnection;
import java.net.URL;
import java.io.BufferedReader;
import java.io.InputStreamReader;
import com.fasterxml.jackson.databind.ObjectMapper;
import com.fasterxml.jackson.databind.JsonNode;
```

2. In the Main agent, create the "error" function that takes a String message as input and shows an alert to the user:

```
public void error(String message) {
    traceln("Error: " + message);
}
```

3. Create the function "getData" in the Main agent that takes an integer input "year", performs the HTTP request, and processes the JSON response:

```
public void getData(int year) {
    try {
        String yearParam = year == 0 ? "latest" : String.valueOf(year);
        String urlString =
"https://datausa.io/api/data?drilldowns=State&measures=Population&year=" +
yearParam;

        URL url = new URL(urlString);
        HttpURLConnection connection = (HttpURLConnection) url.openConnection();
        connection.setRequestMethod("GET");

        int responseCode = connection.getResponseCode();
        if (responseCode != HttpURLConnection.HTTP_OK) {
            error("Unable to fetch data from the API. Response code: " +
responseCode);
            return;
        }

        BufferedReader in = new BufferedReader(new
InputStreamReader(connection.getInputStream()));
        String inputLine;
        StringBuilder content = new StringBuilder();
        while ((inputLine = in.readLine()) != null) {
            content.append(inputLine);
        }

        in.close();
        connection.disconnect();
```

```
            ObjectMapper objectMapper = new ObjectMapper();
            JsonNode rootNode = objectMapper.readTree(content.toString());
            JsonNode dataArray = rootNode.get("data");

            if (dataArray.isEmpty()) {
                error("Empty data received from the API.");
                return;
            }

            for (JsonNode data : dataArray) {
                String state = data.get("State").asText();
                int population = data.get("Population").asInt();
                states.add_State(state, population);
            }
        } catch (Exception e) {
            error("Exception occurred while fetching data: " + e.getMessage());
        }
    }
```

Now, you can call the `getData` function in your Main agent, passing the “year” parameter as an argument. The function will fetch the data, process it, and create new State agents in the “states” population as needed. If there’s an error, it will call the “error” function and pass the corresponding message.